

Pascal-S 编译器使用说明

1. 编译与安装

1.0.1 环境要求

- C++17 兼容的编译器 (g++ 8.0+ 或 Clang 7.0+)
- Flex (2.6+) 和 Bison (3.0+)
- CMake (3.10+) 或 GNU Make
- gcc (用于编译生成的 C 代码)

1.0.2 构建项目

使用项目提供的构建脚本 (推荐):

```
01 bash scripts/build.sh
```

该脚本自动检测 CMake 或 Makefile 并使用合适的构建系统。构建产物为 build/pascc。

如构建脚本不可用, 可手动构建:

```
01 # CMake
02 mkdir -p build && cd build
03 cmake ..
04 make
05
06 # 或直接用 Makefile
07 make
```

2. 基本用法

2.0.1 完整编译

将 Pascal-S 源文件编译为 C 语言目标代码:

```
01 ./build/pascc -i input.pas -o output.c
```

编译成功后, 生成的 C 代码可使用标准 C 编译器编译运行:

```
01 gcc -std=c99 output.c -o program
02 ./program
```

2.0.2 命令行参数

```
01 Usage: pascc -i <input.pas> [-o output.c] [--lex] [--dump-tokens]
02           [--parse] [--semantic] [--dump-annotated-ast]
```

编译器默认执行完整编译流水线（词法 → 语法 → 语义 → 代码生成）。通过以下可选标志可在指定阶段停止：

- `-i <file>`: 指定输入的 Pascal-S 源文件（必需）。
- `-o <file>`: 指定输出的 C 文件路径。省略时仅打印生成消息而不写文件。
- `--lex`: 仅执行词法分析，不输出 Token 流。
- `--dump-tokens`: 执行词法分析并打印 Token 流（格式：Type, Lexeme, Line, Column）。
- `--parse`: 执行词法和语法分析，打印 AST 树形结构后停止。
- `--semantic`: 执行词法、语法和语义分析，打印成功消息或错误信息后停止。
- `--dump-annotated-ast`: 执行到语义分析阶段，打印带类型注解和符号绑定的 AST。

3. 分阶段使用

3.0.1 仅词法分析：查看 Token 流

```
01 ./build/pascc -i test.pas --dump-tokens
```

输出格式为每行一个 Token：TokenType, Lexeme, Line, Column。例如：

```
01 Type, Lexeme, Line, Column
02 Keyword, program, 1, 1
03 Identifier, test, 1, 9
04 Delimiter, ;, 1, 13
05 Keyword, begin, 2, 1
06 Keyword, end, 3, 1
07 Delimiter, ., 3, 4
```

若源文件包含词法错误，错误信息会输出到 `stderr`，同时 Token 流仍会输出到 `stdout`。

3.0.2 仅语法分析：查看 AST 结构

```
01 ./build/pascc -i test.pas --parse
```

输出缩进的 AST 树形结构，展示每个节点的类型和关键信息。例如：

```
01 Parse succeeded.
02 Program
```

03	Block	
04	List	(常量声明列表, 空)
05	List	(类型声明列表, 空)
06	List	(变量声明列表, 空)
07	List	(子程序声明列表, 空)
08	CompoundStmt	(主程序体, 空)

若源文件包含语法错误，输出 Parse error 及行列位置。

3.0.3 仅语义分析：类型检查

```
01 ./build/pascc -i test.pas --semantic
```

执行完整的词法、语法和语义分析。合法程序输出 Semantic analysis succeeded., 非法程序在 stderr 输出具体错误信息。例如：

- 使用未声明变量: Error at 3:5 - Undefined identifier: x
- 类型不匹配: Error at 5:5 - Type mismatch in assignment: expected integer, got boolean
- 数组越界: Error at 5:9 - Array index out of bounds: 0 not in [1, 10]

3.0.4 查看注解后 AST

```
01 ./build/pascc -i test.pas --dump-annotated-ast
```

在语义分析完成后输出 AST，每个节点标注：

- type: 推导出的数据类型 (integer、real、boolean、char)
- isLValue: 是否为可赋值的左值
- [sym: {...}]: 绑定的符号表条目 (名称、种类、作用域层级、类型、是否数组、是否 var 参数)
- [arrayBounds=...]: 数组的声明边界

4. 支持的 Pascal-S 语法

本编译器支持 Pascal-S 语言的以下核心特性：

4.0.1 数据类型

- integer: 整数 (映射为 C int)
- real: 实数 (映射为 C float)
- boolean: 布尔值 (映射为 C int, true = 1, false = 0)
- char: 字符 (映射为 C char)

- `array[low..high] of type`: 数组（支持任意起始下标和多维）
- `record ... end`: 结构体类型（扩展特性，映射为 C `typedef struct`）

4.0.2 声明与语句

- 常量声明: `const name = value`;
- 变量声明: `var name: type`;
- 类型声明: `type Name = record ... end`;
- 赋值语句: `variable := expression`;
- 条件语句: `if cond then stmt else stmt`;
- 循环语句: `while cond do stmt`;
- 计数循环: `for var := start to end do stmt`; 和 `for var := start downto end do stmt`;
- 循环终止: `break`;
- 复合语句: `begin stmt1; stmt2; ... end`;
- 输入输出: `read(varlist)` 和 `write(exprlist)`

4.0.3 子程序

- 过程: `procedure name(params); begin ... end`;
- 函数: `function name(params): returnType; begin ... end`;
- 值参数: `procedure p(x: integer)`;
- 引用参数: `procedure p(var x: integer)`; — 修改实参值
- 不可嵌套定义（仅在全局作用域声明子程序）
- 支持递归调用

4.0.4 表达式

- 算术运算: `+`、`-`、`*`、`/`、`div`、`mod`
- 关系运算: `=`、`<>`、`<`、`<=`、`>`、`>=`
- 逻辑运算: `and`、`or`、`not`
- 函数调用可用作表达式值

4.0.5 类型检查规则

- 变量使用前必须声明，同一作用域内不可重复定义
- 整数可隐式转换为实数（拓宽允许），反向不允许
- 条件表达式（`if/while`）必须为布尔类型
- 数组下标必须为整数且在声明范围内

- 过程/函数调用参数数量和类型必须匹配
- var 参数必须传入可赋值的左值（变量、数组元素、字段）
- break 必须在循环体内

4.0.6 不区分大小写

Pascal-S 标识符不区分大小写。词法分析阶段将所有标识符统一转为小写。因此 MyVar、myvar 和 MYVAR 被视为同一个标识符。

4.0.7 注释

支持 { ... } 风格的注释，且支持嵌套注释（{ { } }）。

5. 开发工具

5.0.1 终端 IDE

项目提供轻量级终端 IDE，支持代码编辑、一键编译和错误导航：

```
01 # 启动空白编辑器
02 bash scripts/run_ide.sh
03
04 # 打开指定文件
05 bash scripts/run_ide.sh my_program.pas
```

快捷键：

- F2 或 Ctrl+S：保存文件
- F5：编译当前文件（含词法、语法、语义分析报告）
- F9：编译并运行（生成 C → gcc 编译 → 执行）
- Ctrl+Q：退出
- 命令模式（Ctrl+X）：:w 保存，:q 退出，:wq 保存退出

5.0.2 编译过程可视化

TUI 可视化工具实时展示编译器各阶段的处理进度：

```
01 bash scripts/run_tui.sh -i test.pas
02 bash scripts/run_tui.sh -i test.pas --log-level debug
```

6. 常见问题

编译错误：“pascc not found” 确保已运行 `bash scripts/build.sh` 完成构建。构建产物位于 `build/pascc`。

生成的 C 代码编译失败 确保使用 C99 或更高标准的 gcc: `gcc -std=c99 output.c -o program`。Pascal 的 `real` 类型映射为 C `float`，如涉及数学函数需链接 `-lm`。

语法分析报语法错误但看起来是正确的 **Pascal** 确认文件以 `.pas` 后缀命名，且程序以 `program name; ... end.` 格式书写（末尾须有点号 `.`）。

标识符内有下划线被识别为非法 Pascal-S 标识符允许字母、数字和下划线，以字母或下划线开头。检查是否有特殊字符混入。

GCC 报 `bracket nesting level exceeded` 错误 部分极限测试用例（如 `56_long_code2.pas`）的嵌套深度超过 gcc 默认 256 层限制。这是已知行为，不影响 `pascc` 本身对该程序的正确编译。如需通过 gcc 编译，可使用 `-fbracket-depth=N` 放宽限制。